PUMA: Client Resource-Aware Adaptive Pruning with Heterogeneous Sub-model Aggregation

R. Veiga*†, J. Sousa*, B. Barreto†, L. Bastos‡, D. Rosário*, S. Sargento†, and E. Cerqueira*
* Dept. of Computer Science - Federal University of Pará, Brazil
† Instituto de Telecomunicações, University of Aveiro, Portugal
‡ Federal University of South and Southeast of Pará (UNIFESSPA), Brazil
Emails: rafael.teixeira.silva@icen.ufpa.br, john.sousa@itec.ufpa.br, bernardo.barreto@ua.pt
bastos.lucas@unifesspa.edu.br, susana@ua.pt, {denis,cerqueira}@ufpa.br.

***Abstract*—Federated Learning (FL) has emerged as a key and decentralized approach to training Machine Learning (ML) models locally on edge devices while preserving client data privacy. However, in scenarios with heterogeneous edge devices and dynamic networks, making FL practical is a hard task. This paper introduces PUMA, an adaptive pruning-based learning approach that combines model pruning, quantization, and sub-model aggregation based on client resource-awareness, improving FL performance and accuracy in heterogeneous scenarios, even for devices with severe computational or bandwidth constraints. The evaluation results show that PUMA outperforms the baseline approaches in terms of accuracy, training time, and model size.**

***Index Terms*—Federated Learning, Pruning, Adaptive Pruning, Personalized**

I. Introduction

Federated Learning (FL) has emerged as a valuable approach for intelligent environments, mainly due to its collaboratively trained Machine Learning (ML) models without sharing raw data from clients [1]. FL not only aims to protect data privacy, but also improves data generalization by training clients about sensitive information [2]. However, FL does not perform well in heterogeneous scenarios, where clients may have different data, computational, and network resources [3]. Therefore, a personalized approach for each client is required to improve FL performance in heterogeneous environments.

Personalized Federated Learning (pFL) reduces the burden on clients by tailoring models to the specific needs and data distribution of each client, effectively combating system and statistical heterogeneity [4]. pFL improves both local performance and global robustness by allowing clients to adjust local models based on the importance of the layer and the unique characteristics of the data [5]. A key factor in pFL is determining which model parameters are expendable, which is achieved by measuring each layer's contribution to predictive accuracy relative to its computational cost. In this way, clients can selectively retain only the most informative weights [5]. Hence, pFL determines how much a client needs to have a minor model, tailoring the time and processing required to complete a round.

Removing unnecessary model parameters (*i.e.*, model pruning) is an effective way to adapt pFL in a heterogeneous environment. Personalized model pruning based on client resource-awareness (*e.g.*, current bandwidth conditions, local training data, and processing of FLOPs of the model) offers a viable way to make models more compact and easier to store or transmit [6]. For instance, structured pruning removes entire filters or layers, avoiding the disadvantages of unstructured pruning, such as individual weight removal, as well as the need for additional storage for metadata, such as compressed sparse rows or binary masks. Hence, structured pruning preserves a dense model format compatible with limited hardware, directly cutting both training and latency times [7].

Pruning-based approaches reduce model overhead but ignore dynamic bandwidth and real-time client communication rates [6], [8], [9]. These approaches do not account for the creation of sub-models during pruning, leading to models with different shapes that require a proper aggregation method. In contrast, traditional FL does not perform well in mapping to the original model. Furthermore, current methods frequently lack the flexibility to aggregate heterogeneous sub-models efficiently, requiring all clients to train the same architecture and often forcing faster devices to remain idle, while waiting for slower stragglers to complete training [10]. Hence, it is important to design an approach that simultaneously deals with client resource-awareness and aggregates heterogeneous sub-models, which remains an open challenge not fully addressed by existing FL pruning schemes.

In this paper, we introduce an adaptive pruning approach combined with heterogeneous sub-model aggregation for pFL in heterogeneous environments, called PUMA. PUMA performs quantization and pruning methods to adjust the ML model based on the awareness of client resources, improving the FL efficiency and accuracy in heterogeneous scenarios. To enable aggregation with updates from other differently pruned models, the server applies a Pad function to expand the model back to the dimensions of the global model for sub-model aggregation. The evaluation results show that PUMA outperforms the baseline approaches. For example, PUMA takes less than 50% of the convergence time, while achieving almost 10% more accuracy than the baseline approach.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III describes our approach to adaptive pruning aggregation in both the server and client models. Section IV explores the simulation scenario and results. Finally, Section V concludes the paper and outlines the directions for future work.

II. Related Work

Xu *et al.* [8] developed the FL-PQSU approach to support resource-constrained Internet of Things (IoT) devices, primar-

ily for big data analytics. FL-PQSU aims to reduce computational, storage, and communication overhead by combining structured pruning, weight quantization, and selective client-side model updates. However, FL-PQSU did not account for dynamic client values such as data and transmission rates or hardware differences between clients, which are critical for heterogeneous edge environments.

Jiang *et al.* [6] presented PruneFL to address computational and communication challenges, specifically focusing on client devices. PruneFL utilized statistics from client data and systems to reduce model size, minimize latency, and improve resource consumption by scaling pruning limits. Nevertheless, PruneFL did not consider the actual rate of client communication or the impact of hardware heterogeneity on personalized training.

Jiang *et al.* [9] proposed an adaptive model pruning framework for FL, called FedMP, which addressed computational and communication heterogeneity among edge workers. FedMP mitigated the straggler effect by assigning personalized pruning ratios to workers based on their individual capabilities and employed a Multi-Armed Bandit-based online learning algorithm to determine these ratios. However, FedMP did not account for wireless channel dynamics or the specific trade-off between local training time and communication costs required for optimized pFL.

Chen *et al.* [11] proposed an adaptive model pruning (AMP-FL) framework for wireless FL that jointly optimizes device-level pruning and radio resource block allocation. AMP-FL assigned each scheduled device a compact sub-model tailored to its capacity to mitigate the straggler effect. However, AMP-FL required the edge server to maintain full-dimensional gradient buffers for every device, leading to a memory overhead that scaled linearly with the number of clients and failed to optimize the training time per-round efficiently.

Fan *et al.* [10] addressed client idle time by sending each client a differently pruned sub-model sized to match expected completion times. This strategy ensured that faster clients did not remain idle during the round. Nevertheless, the strategy did not explicitly balance the precision of the personalized model with the heterogeneous computational capabilities of the hardware, a gap that often results in suboptimal learning efficiency in practical edge deployments.

Nevertheless, existing approaches do not consider client hardware differences or client resource awareness when calculating the pruning percentage. The pruning approach causes sub-models, that need to be treated correctly, to aggregate the minor tailored models into the global model, considering that resources are needed to balance the delivery time of clients in transmission and training time. To the best of our knowledge, only PUMA considers all these features in an adaptive running approach.

III. Adaptive Pruning Approach for pFL

This section introduces the PUMA approach, which provides structured adaptive pruning and sub-model aggregation based on device resource-awareness. Balancing the pruning rate based on clients' resource awareness in each round enables slow clients to participate in their tailored sub-model aggregation sooner, reducing the overall time to break bottlenecks. In the following subsections, we introduce the system model and operational details of PUMA.

A. Scenario overview

Figure 1 shows the typical workflow of pFL in a distributed setting, where the standard FL procedure consists of three key steps per communication round t: selection of clients, training, and aggregation. We consider a scenario with devices n, denoted by $\mathcal{P} = \{p_1, \ldots, p_n\}$, where each device is uniquely identified by an index $i \in [1, n]$. Each round t begins with the selection of the client, which chooses a subset $\mathcal{C} \subseteq \mathcal{P}$ of devices (clients) that possess the most valuable data samples, potentially omitting clients whose data no longer contribute meaningfully to model training. The server then transmits the current global model parameters M_θ to all participating clients for classification or prediction. The ML model is a Neural Network with L layers, represented as $\mathcal{L} = \{l_1, l_2, \ldots, l_L\}$.

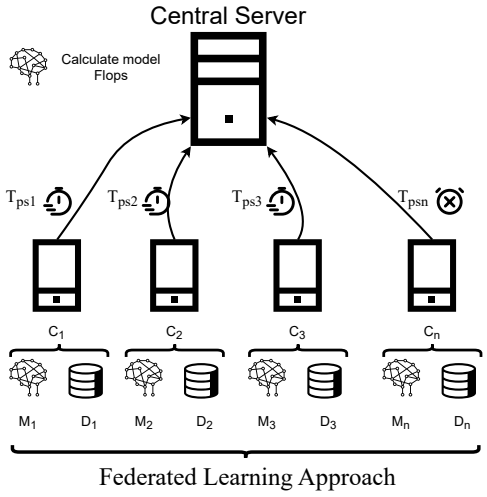


Fig. 1. Scenario overview for Traditional FL

Each device p_i has a local dataset D_i that varies in size and follows a Non-Independent and Identically Distributed (non-IID) distribution of features and labels. This dataset comprises $|D_i|$ samples, where each sample k consists of a feature vector $x_{k,i}$ and its corresponding label $y_{k,i}$. At the beginning of the round, the server starts initial client resource measurements and a timer T_{ps} to compute each client's training time, as well as establishes the server-client connection to calculate each client's bandwidth B_i based on the received data amount.

During the training step, each client p_i refines the global model M by updating its local parameters θ_i. Each client minimizes a local loss function $L_i(\theta)$ over a specified number of epochs E, as defined in Eq. 1 of Adaptive Local Aggregation for Personalized Federated Learning (FedALA) [12]. Here, $f(x_{k,i}; \theta)$ denotes the model's prediction for input $x_{k,i}$, and ℓ represents the task-specific loss function, such as cross-entropy or mean squared error. Using the diversity of local datasets, FedALA improves the robustness of the global model and reduces the impact of smaller or less representative datasets. Its weighted averaging mechanism ensures that clients with larger

datasets contribute proportionally more to the global update, effectively addressing data imbalances and promoting convergence in non-IID data environments. This cycle repeats over multiple communication rounds until the model converges.

$$L_i(\theta) = \frac{1}{|D_i|} \sum_{(x_{k,i}, y_{k,i}) \in D_i} \ell(f(x_{k,i}; \theta), y_{k,i}), \tag{1}$$

Following local training, each client transmits its updated parameters M_i^t to the central server for aggregation, refining the global model. The aggregation step completes the communication round by combining the model updates of the selected clients $\mathcal{C}$ to produce the new global model M_g^{t+1}.

B. PUMA Operations

PUMA implements a two-stage pipeline designed to optimize pFL performance in heterogeneous environments. During the initial calibration round, each client C_i executes local training over E epochs, measuring its training time T^i_{train}. The server concurrently measures the communication time T^i_{comm} to receive the updates from each client and calculates the effective bandwidth B_i. These client-specific metrics B_i and T^i_{train} are stored in a dictionary $\mathcal{D}$. The server then computes the target synchronization time T_{target} as the maximum total time observed between all clients, establishing the deadline for subsequent communication rounds.

In all subsequent adaptive rounds, PUMA starts its core optimization phase. For each client C_i, the server retrieves its profile from $\mathcal{D}$ and calculates the time T^i_{full} required to process the full model. It then computes a personalized pruning ratio p_i as $T_{\text{target}}/T^i_{\text{full}}$, capping it at 1.0. Using this p_i, the server invokes the prune function to generate a tailored sub-model M_i and transmits it to the client for local training. After training, the client returns its local parameters θ_i. To enable aggregation with updates from other differently pruned models, the server applies a Pad function to expand θ_i back to the dimensions of the global model, producing $\hat{\theta}_i$.

The Pad function zero-fills the pruned parameter tensor θ_i back to the full global model dimensions. In this sense, channels or filters removed during pruning are re-inserted as zero-valued entries, and thus the padded update $\hat{\theta}_i$ has an identical shape to the global model M, and can be element-wise combined during aggregation. The algorithm 1 executes the Aggregate function on the server to combine all padded updates $\{\hat{\theta}_i\}$ into a new global model M for the next round. This cyclical process including adaptive pruning, personalized sub-model distribution, local training, and heterogeneous aggregation, constitutes the entire PUMA pipeline. By dynamically adjusting model complexity to align client completion times with T_{target}, Algorithm 1 mitigates the straggler effect, minimizes idle waiting, and accelerates convergence in non-IID and resource-heterogeneous FL.

The heterogeneous aggregation of sub-models is formalized through a sample-count matrix $H \in \mathbb{Z}^d_{\geq 0}$, where d denotes the total number of parameters in the global model, initialized to all zeros at the start of training. The entry $H[k]$ accumulates the total number of training samples that have contributed to the parameter k in all rounds and clients; it serves as the denominator on the weighted average, ensuring that the parameters updated by more data receive proportionally smaller increases per-sample. Figure 2 shows that during aggregation, the update from a client c whose sub-model has, for a given layer, I_c input and O_c output channels that correspond to a specific sub-matrix of the global parameter space. The server increments the heat values in the corresponding $I_c \times O_c$ sub-matrix of H by the client's local dataset size $|D_c|$. The server then aggregates updates by performing a weighted average: each parameter update is scaled by $|D_c|$ and divided by its accumulated heat value in H. This ensures that parameters updated by multiple clients are averaged proportionally to their contributions, while those updated by only one client retain their local values.

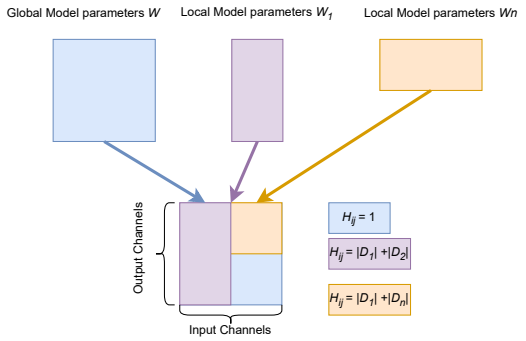


Fig. 2. Personalized Pruning Heterogeneous Sub-model Aggregation

The training time of a model is proportional to its local dataset size and the FLOPs needed to complete training. Let $\beta = \sum \beta_l$ be the total FLOPs of the full model. After applying a pruning rate p^c (the fraction of parameters and FLOPs removed), the remaining FLOPs are $(1 - p^c)\beta$. The time per FLOP, t_{FLOP}, is estimated from the client's previous training round using Eq. 2.

$$t_{\text{FLOP}} = \frac{\text{training_time}}{|\mathcal{D}^c| \cdot \beta} \tag{2}$$

While pruning reduces the number of computations, it does not inherently reduce the bit-width of parameters transmitted over the network. To further reduce communication overhead, PUMA optionally applies $int8$ quantization, compressing each parameter to 1 byte. Formally, let $\alpha = \sum \alpha_l$ be the total number of parameters in the full model. The pruned model retains $(1 - p^c)\alpha$ parameters, requiring $(1 - p^c)\alpha \cdot b$ bytes for transmission, where $b = 1$ byte/parameter under $int8$ encoding. Given client bandwidth B^c (bytes/second), the communication latency is given by Eq. 3, which gives a scalability of usage of that equation in performance and balance.

$$\frac{(1 - p^c)\,\alpha \cdot b}{B^c} = (1 - p^c) \underbrace{\frac{\alpha \cdot b}{B^c}}_{\text{full communication cost}} \tag{3}$$

Given the time budget L_{target} for a complete client round (training plus transmission), the pruning rate p^c is derived

by requiring that the total client time $L(p^c)$ does not exceed L_{target}. Substituting the training time model (Eq. 2) and the communication latency model (Eq. 3), and solving for p^c yields Eq. 4.

$$p^c = 1 \; - \; \frac{L_{\text{target}}}{|\mathcal{D}^c|\,\beta\,t_{\text{FLOP}} + \dfrac{\alpha \cdot b}{B^c}} \tag{4}$$

The computed pruning rate is clipped to the interval $[0.2,\, 0.85]$ to enforce a minimum pruning of 20%, which guarantees a meaningful model reduction, and a maximum of 85%, which prevents excessive capacity loss that could harm model accuracy. After computing the clipped rate, the server optionally applies a history-based correction derived from the client's training time behaviour, as given in Eq. 5.

$$p^c \leftarrow \begin{cases} p^c + 0.05, & \text{if training_time}_c > 1.5 \times \text{last_time}_c \\ p^c + 0.03, & \text{if training_time}_c > 1.2 \times L_{\text{target}} \\ p^c, & \text{otherwise.} \end{cases} \tag{5}$$

The thresholds $1.5\times$ and $1.2\times$, along with the increments 0.05 and 0.03, were selected via a grid search over the ranges $\{1.1, 1.2, 1.3, 1.5, 2.0\}$ and $\{0.01, 0.03, 0.05, 0.10\}$, respectively, minimizing mean per-round training time over 25 rounds on a held-out heterogeneous client set. The chosen values yielded the best trade-off between correction speed and stability; larger increments caused oscillation in the pruning rate, while smaller ones were insufficient to recover from persistent stragglers within a few rounds.

Algorithm 1 shows the PUMA pipeline for computing the client-prune percentage, leveraging resource awareness and sub-model aggregation. In the initialization phase of FL on lines 1 to 10, the server collects each client's resources (bandwidth, data, and model parameters) into a dictionary. It sets the target average time T_{target}. For every round after round 0, PUMA computes pruning for each client using Eq. (4) based on its dictionary entry. If a client is not selected, Eq. (5) determines a new pruning for that client. Finally, the server aggregates the received sub-models.

Algorithm 1: PUMA Adaptive Pruning FL

1 Initialize client information dictionary $\mathcal{D} \leftarrow \emptyset$;
2 **for** $r \leftarrow 0$ **to** *convergence* **do**
3 **if** $r = 0$ **then**
4 **foreach** *client* $C_i \in \mathcal{C}$*in parallel* **do**
5 $M \rightarrow C_i$;
6 $\theta_i, T^i_{\text{train}} \leftarrow \text{Train}(C_i, M, E)$;
7 $\theta_i \rightarrow$ Server;
8 $B_i \leftarrow S_{\text{full}}/T^i_{\text{comm}}$; $\quad \mathcal{D}[i] \leftarrow (B_i, T^i_{\text{train}})$;
9 **end**
10 Set target time $T_{\text{target}} \leftarrow \max_i(T^i_{\text{train}} + T^i_{\text{comm}})$;
11 **else**
12 **foreach** *client* $C_i \in \mathcal{C}$ *in parallel* **do**
13 Retrieve B_i and T^i_{train} from $\mathcal{D}[i]$;
14 Compute total full time: $T^i_{\text{full}} \leftarrow T^i_{\text{train}} + S_{\text{full}}/B_i$;
15 Compute pruning ratio: $p_i \leftarrow T_{\text{target}}/T^i_{\text{full}}$;
16 **if** $p_i > 1$ **then**
17 $p_i \leftarrow 1$;
18 **end**
19 $M_i \leftarrow \text{Prune}(M, p_i)$;
20 $M_i \rightarrow C_i$;
21 $\theta_i \leftarrow C_i(M_i, E)$ train local sub-model $\hat{\theta}_i \leftarrow \text{Pad}(\theta_i)$ padding
22 **end**
23 Aggregate updates: $M \leftarrow \text{Aggregate}(M, \{\theta_i\}_{i \in \mathcal{C}})$;
24 **end**
25 **end**
26 **return** M;

IV. Evaluation

This section presents the simulation setup used to evaluate the performance and efficiency of PUMA compared to the Baseline approaches. We first describe the simulated FL scenario, including the underlying framework, the characteristics of the database, and the simulation parameters. We also present the obtained results, focusing on the accuracy and loss metrics for the global model.

A. Simulation Setup

We consider an open-source Framework to support multiple pFL methods based on PFLlib[1], which is a widely used framework. Its architecture allows seamless customization and personalization of models for individual clients, making it well-suited to our objectives, whether used in Docker or local environments. Our simulation code and Framework are available on GitHub[2]. Table I outlines the key simulation parameters used in our experiments.

TABLE I
Simulation Parameters

Description	Values
Number of Clients	8 Clients
Training clients	100% of total number of clients
Default dataset	CIFAR-10
Local epochs	1 epoch per round
Number of classes	10 classes
Rounds	25
Base model size	3.5
Used model	CNN
Default Bandwidth	1GB
Pruning Methods	PUMA, SNIP and NISP

The experiments consider the CIFAR-10 datasets, where each client performs one local epoch per round, ensuring consistent local training updates between all participants. This dataset is suitable for evaluating FL algorithms in scenarios requiring higher levels of granularity and complexity in image

[1]PFLlib available at $github.com/TsingZ0/PFLlib$

[2]PUMA is available at $github.com/VeigarGit/FL_SerwerPuma$

classification. We consider data with non-IID characteristics to introduce data heterogeneity, ensuring a realistic representation of data distribution as commonly expected in many FL applications. We used the Dirichlet distribution with a $\beta = 0.2$ concentration parameter, as described in [13].

We consider a widely recognized deep learning architecture (*i.e.*, a VGG, CNN) and a straightforward benchmark ML model [14]. It comprises multiple stacked 3x3 convolutional layers followed by max-pooling layers, which gradually reduce spatial dimensions while increasing the depth of feature maps. This structure allows for efficient feature extraction. The heavy reliance on fully connected layers in CNNs increases computational cost and the risk of overfitting, especially on small datasets. However, they remain benchmarks due to their simplicity and effectiveness in image classification.

We compare PUMA's performance with three existing approaches: single-shot network pruning (SNIP) [6], NISP [15], and Default FL. Specifically, SNIP identifies the most critical connections in a neural network, enabling structured pruning while maintaining performance. NISP evaluates the importance of neurons by considering their contributions to the final output. The Default FL (FedALA Only) is the traditional FL approach without pruning, but it uses $int8$ quantization. Finally, PUMA provides adaptive and structured pruning based on delivery time and round-trip threshold time, slower processing, and bandwidth capabilities, thus contributing to the global model with faster convergence, as introduced in Section III. In addition, we evaluated two variants of PUMA: PUMA 100 MB/s and PUMA 500 MB/s, which enforce a bandwidth constraint, demonstrating the method's adaptability to different resource limitations and their impact on pruning effectiveness.

Our evaluation focuses on three critical performance metrics as follows: *(i)* Round Time, *(ii)* the volume of data transmitted from the server to clients, and *(iii)* the client-side accuracy relative to the amount of data sent by the server. Through this structured comparative analysis, our aim is to highlight the strengths of our approach in achieving superior efficiency and scalability without compromising predictive accuracy.

B. Results

Figure 3 shows the total model size transmission for Default FL and PUMA in a scenario with quantized and non-quantized models. By analyzing the results, we can observe that model quantization reduces the model size and accelerates inference by converting 32-bit floating-point values to 8-bit integers, making it faster and using less space. For instance, the Default FL sends an average of 670.8 MB in total after 25 rounds without quantization, while the Default FL with quantization reduces the transmission by half, reaching around 168.5 MB in total for the simulation. On the other hand, PUMA Default (*i.e.*, without quantization) transmits 116 MB, but PUMA with quantization sends only 29.9 MB on average. Hence, this is a valuable technique for solving transmission problems, making it especially useful for pruning.

Figure 4 shows the accuracy over the cumulative time of the pruning approaches. We can first evaluate Default FL and NISP, which have the highest running time in our simulation, and also achieve an accuracy of over 85% in the final rounds. This is because the clients have an entire model to process

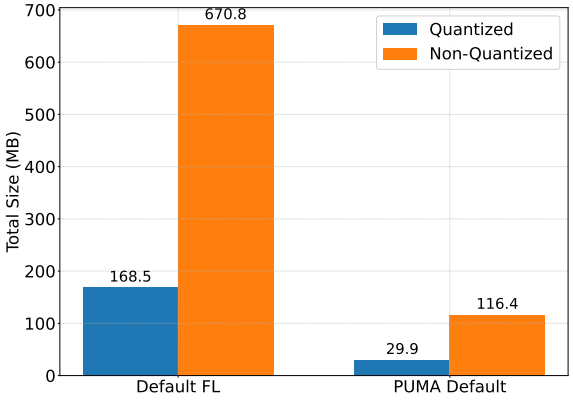


Fig. 3. Quantization importance measurements

in both approaches, Default FL and NISP. Using quantization with $int8$, NISP shows that the most significant bottlenecks are not related to transmission, but to the hardware sequence. Therefore, neither the entire model nor the non-structural model are compatible with the expected improvement.

SNIP reduces the training time to 1500 seconds compared to both Default FL and NISP, as it uses structural pruning to produce a faster model with a modest amount of data. The PUMA Default approach shows that the pruning method provides a higher accuracy with a shorter training time. For instance, the default PUMA reaches an accuracy of around 85% in less than 1500 seconds, almost half of the Default FL. Hence, our approach achieves these results through the use of structure pruning, which reduces both the parameter connections and the size of the model. This issue improves training time and transmission, by balancing the pruning percentage. Despite achieving the same evaluation, the PUMA 100MB/s and PUMA 500MB/s reached almost the same accuracy result.

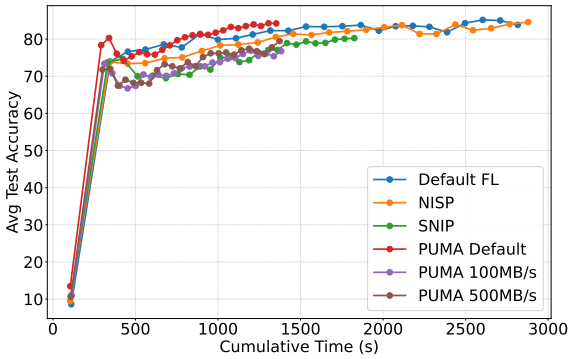


Fig. 4. Accuracy by time measurements

Table II shows the average communication between the server and client, including how much data was sent per client (in MB) and the round-trip time. Default FL sends 0.810 MB per client because it does not use pruning. SNIP and NISP send less data, *i.e.*, 0.344 MB and 0.317 MB per client, respectively. However, their round-trip times are 73.04 seconds and 112.64 seconds, respectively. NISP takes longer because its pruning method is not structural. This shows that the real delay occurs

during local processing (on the hardware), not in sending data. SNIP improves round-trip time thanks to its pruning style.

PUMA Default and 500Mb/s versions show further improvement by using structural pruning to tailor the model better, sending only 0.144 MB and 0.148 MB, while maintaining accuracy with 54.13 and 54.93, respectively. To handle the 100 MB/s bandwidth limit, PUMA slows down slower clients while giving faster clients (likely those with less data) larger models. This approach aims to speed up training by sending only 0.153 MB on average, but accuracy suffers. While structural pruning helps compress the model, it introduces a clear trade-off between model size and accuracy.

TABLE II
AVERAGE SERVER-CLIENTS COMMUNICATION DATA

Method	**Average Model per client**	**Round time**
Default FL	0.810 MB	115.17s
NISP	0.317 MB	112.64s
SNIP	0.344 MB	73.04s
PUMA Default	0.144 MB	54.13s
PUMA 500mb	0.148 MB	54.92s
PUMA 100mb	0.153 MB	55.32s

V. CONCLUSION

This paper proposed PUMA as a pFL approach to enhance client participation in aggregation by using sub-model aggregation and pruning to speed up training rounds, taking into account clients' resource awareness in heterogeneous scenarios. PUMA improves processing and round times for each client by considering potential bottlenecks and resource-awareness factors, such as data volume, current bandwidth conditions, and model flops, which are used as weights to calculate the pruning percentage for each client. The evaluation results demonstrate that PUMA outperforms the Baselines and state-of-the-art pruning approaches, such as NISP and SNIP, taking less than half the time to run the exact simulation, and achieving in the PUMA Default almost 10% more accuracy than NISP, reaching an evaluated accuracy of around 85%. These results highlight the robustness and efficiency of PUMA in pruning and improving delivery time in FL-pruned model environments.

Future work includes a deep analysis and evaluation of PUMA with various compression strategies in heterogeneous and real environments. For instance, this will involve aggregations through entropy-based delta compression to help eliminate redundancies among client parameters.

ACKNOWLEDGMENT

This work was supported by the Ministry of Science, Technology and Innovation, with resources from Law No. 8,248 of October 23, 1991, within the scope of PPI-SOFTEX, coordinated by Softex and published as Cognitive Architecture (Phase 3), DOU 01245.003479/2024-10. This research is also part of the INCT of Intelligent Communications Networks and the Internet of Things (ICoNIoT) funded by CNPq (proc. 405940/2022-0) and Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 88887.954253/2024-00. Furthermore, this work was funded by the National Institute of Science and Technology in Artificial Intelligence Applied to Smart and Sustainable Cities in the Brazilian Amazon (INCT IAmazônia), grant nº 409001/2024-4. Finally, the authors would like to acknowledge the support from CNPq under grant number 405940/2022-0. This work is also funded by PRR–Plano de Recuperação e Resiliência and by the NextGeneration EU funds at Universidade de Aveiro, through the scope of the Agenda for Business Innovation "ATT– Agenda Mobilizadora Acelerar e Transformar o Turismo" (Project no. 47 with the application C645192610-00000060).

REFERENCES

[1] P. Kairouz, H. B. McMahan, B. Avent, and et al., "Advances and open problems in federated learning," *Foundations and trends in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[2] B. Hu, X. Huang, K. Zhang, F. Wu, C. Sun, T. Cui, and Y. Zhang, "Enhanced federated reinforcement learning for mobility-aware node selection and model compression," in *IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2023, pp. 158–163.

[3] B. Martins, A. M. Souza, D. Rosário, C. A. Astudillo, E. Cerqueira, and L. Villas, "Partial training mechanism to handle the impact of stragglers in federated learning with heterogeneous clients," in *IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2024, pp. 1–6.

[4] M. Ye, X. Fang, B. Du, P. C. Yuen, and D. Tao, "Heterogeneous federated learning: State-of-the-art and research challenges," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–44, 2023.

[5] A. M. de Souza, F. Maciel, J. B. da Costa, L. F. Bittencourt, E. Cerqueira, A. A. Loureiro, and L. A. Villas, "Adaptive client selection with personalization for communication efficient federated learning," *Ad Hoc Networks*, vol. 157, p. 103462, 2024.

[6] Y. Jiang, S. Wang, V. Valls, B. J. Ko, W.-H. Lee, K. K. Leung, and L. Tassiulas, "Model pruning enables efficient federated learning on edge devices," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 10 374–10 386, 2022.

[7] N. Yan, Y. Li, J. Chen, X. Wang, J. Hong, K. He, and W. Wang, "Efficient and straggler-resistant homomorphic encryption for heterogeneous federated learning," in *IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2024, pp. 791–800.

[8] W. Xu, W. Fang, Y. Ding, M. Zou, and N. Xiong, "Accelerating federated learning for iot in big data analytics with pruning, quantization and selective updating," *IEEE Access*, vol. 9, pp. 38 457–38 466, 2021.

[9] Z. Jiang, Y. Xu, H. Xu, Z. Wang, J. Liu, Q. Chen, and C. Qiao, "Computation and communication efficient federated learning with adaptive model pruning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 3, pp. 2003–2021, 2023.

[10] W. Fan, K. Yang, Y. Wang, C. Chen, and J. Li, "Data-free adaptive structured pruning for federated learning," *The Journal of Supercomputing*, vol. 80, no. 13, pp. 18 600–18 626, 2024.

[11] Z. Chen, W. Yi, H. Shin, and A. Nallanathan, "Adaptive model pruning for communication and computation efficient wireless federated learning," *IEEE Transactions on Wireless Communications*, vol. 23, no. 7, pp. 7582–7598, 2023.

[12] J. Zhang, Y. Hua, H. Wang, T. Song, Z. Xue, R. Ma, and H. Guan, "Fedala: Adaptive local aggregation for personalized federated learning," in *AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11 237–11 244.

[13] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 10 713–10 722.

[14] W. Hussain, M. F. Mushtaq, M. Shahroz, U. Akram, E. S. Ghith, M. Tlija, T.-h. Kim, and I. Ashraf, "Ensemble genetic and cnn model-based image classification by enhancing hyperparameter tuning," *Scientific Reports*, vol. 15, no. 1, p. 1003, 2025.

[15] R. Yu, A. Li, C.-F. Chen, J.-H. Lai, V. I. Morariu, X. Han, M. Gao, C.-Y. Lin, and L. S. Davis, "Nisp: Pruning networks using neuron importance score propagation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.